STATISTICS & MACHINE LEARNING · LESSON 15 OF 30

Cross-Validation and Hyperparameter Tuning: K-Fold, GridSearchCV, RandomizedSearchCV and Optuna

Estimate performance reliably and tune models without fooling yourself. Hold-out vs k-fold, stratified, grouped and time-series cross-validation, cross_validate with several metrics, GridSearchCV and RandomizedSearchCV over pipelines, nested cross-validation, Bayesian optimisation with Optuna, and the common leakage traps.

ML classes 6–7 · ✦ scikit-learn 1.7+ · Optuna 4 · Intermediate · ⏱ 110 min

By the end of this lesson you can

- Explain why one train/test split is noisy and how k-fold cross-validation helps
- Choose stratified, group and time-series splitters for the problem
- Tune pipeline hyperparameters with GridSearchCV and RandomizedSearchCV and read cv_results_
- Use nested cross-validation for an unbiased estimate of a tuned model
- Recognise and prevent leakage during validation

Datasets used

`bank_loans.csv` `daily_sales.csv`

One split is noisy

Python

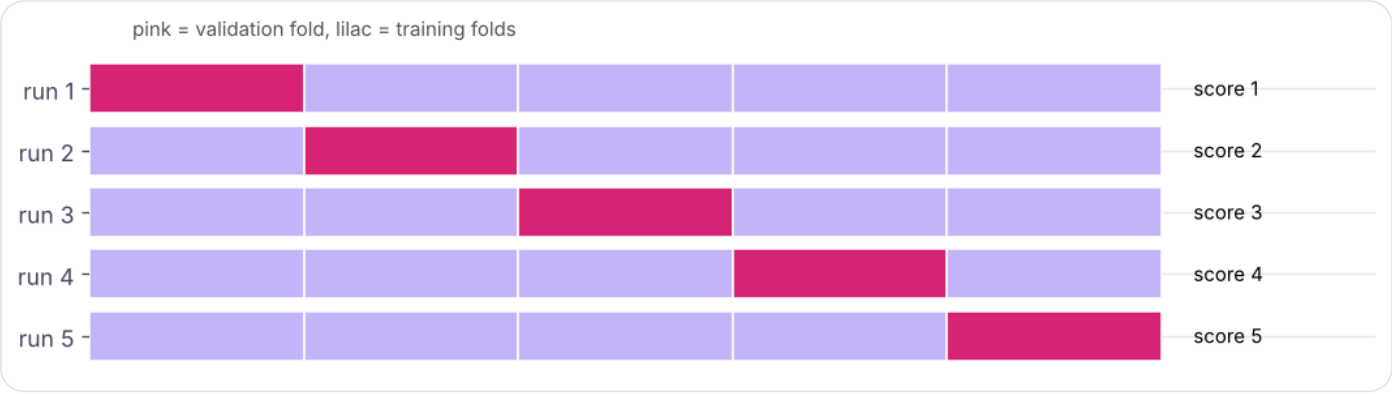
```
import pandas as pd, numpy as np
from sklearn.model_selection import train_test_split, cross_val_score, cross_validate,
StratifiedKFold, KFold, GridSearchCV, RandomizedSearchCV
from sklearn.pipeline import make_pipeline, Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import roc_auc_score
b = pd.read_csv("bank_loans.csv").dropna(subset=["default"])
feats = ["age", "ed", "employ", "address", "income", "debtinc", "creddebt", "othdebt"]
X, y = b[feats], b.default.astype(int)
pipe = make_pipeline(StandardScaler(), LogisticRegression(max_iter=1000))
aucs = []
for seed in range(20):
    Xa, Xb, ya, yb = train_test_split(X, y, test_size=0.25, stratify=y, random_state=seed)
    aucs.append(roc_auc_score(yb, pipe.fit(Xa, ya).predict_proba(Xb)[:, 1]))
print(f"20 different random splits: test AUC from {min(aucs):.3f} to {max(aucs):.3f} (mean
{np.mean(aucs):.3f})")
```

OUTPUT

```
20 different random splits: test AUC from 0.745 to 0.865 (mean 0.801)
```

With 700 rows, the test score depends a lot on **which** rows land in the test set. Cross-validation averages over several splits.

K-fold cross-validation



5-fold cross-validation: each fold is the validation set once; the score is the average of 5 runs.

Python

```python
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=0)
scores = cross_val_score(pipe, X, y, cv=cv, scoring="roc_auc")
print("fold AUCs:", scores.round(3), f"| mean {scores.mean():.3f} ± {scores.std():.3f}")
res = cross_validate(pipe, X, y, cv=cv, scoring=["roc_auc", "accuracy", "recall",
"precision"], return_train_score=True)
print({k: round(float(v.mean()), 3) for k, v in res.items() if "time" not in k})
```

OUTPUT

```
fold AUCs: [0.767 0.862 0.746 0.83  0.811] | mean 0.803 ± 0.042
{'test_roc_auc': 0.803, 'train_roc_auc': 0.817, 'test_accuracy': 0.749,
'train_accuracy': 0.762, 'test_recall': 0.48, 'train_recall': 0.49, 'test_precision':
0.667, 'train_precision': 0.68}
```

Report the **mean and spread**. Compare train and validation scores to spot overfitting.

Splitter	Use
`KFold`	Regression, shuffled independent rows
`StratifiedKFold`	Classification: keeps class balance per fold (default for classifiers)
`GroupKFold`	Several rows per customer/patient/store: a group never appears in both train and validation
`TimeSeriesSplit`	Time order: always train on the past, validate on the future
`RepeatedStratifiedKFold`	Small data: repeat k-fold with different shuffles
`LeaveOneOut`	Tiny datasets (expensive)

Time-series cross-validation

Python

```
from sklearn.model_selection import TimeSeriesSplit
from sklearn.ensemble import HistGradientBoostingRegressor
from sklearn.metrics import mean_absolute_error
d = pd.read_csv("daily_sales.csv", parse_dates=["date"])
f = pd.DataFrame({"dow": d.date.dt.dayofweek, "month": d.date.dt.month, "t":
np.arange(len(d)), "lag7": d.units_sold.shift(7), "lag14":
d.units_sold.shift(14)}).dropna()
yt = d.units_sold.loc[f.index]
for i, (tr, va) in enumerate(TimeSeriesSplit(n_splits=4, test_size=90).split(f)):
    m = HistGradientBoostingRegressor(random_state=0).fit(f.iloc[tr], yt.iloc[tr])
    print(f"fold {i + 1}: train to {d.date[f.index[tr[-1]]].date()} ({len(tr)} days) →
validate {d.date[f.index[va[0]]].date()}–{d.date[f.index[va[-1]]].date()} | MAE
{mean_absolute_error(yt.iloc[va], m.predict(f.iloc[va])):.1f}")
shuffled = cross_val_score(HistGradientBoostingRegressor(random_state=0), f, yt,
cv=KFold(5, shuffle=True, random_state=0), scoring="neg_mean_absolute_error").mean()
print(f"shuffled KFold MAE {-shuffled:.1f}: optimistic, because it trains on the future")
```

OUTPUT

```
fold 1: train to 2025-01-05 (1087 days) → validate 2025-01-06–2025-04-05 | MAE 24.3
fold 2: train to 2025-04-05 (1177 days) → validate 2025-04-06–2025-07-04 | MAE 30.8
fold 3: train to 2025-07-04 (1267 days) → validate 2025-07-05–2025-10-02 | MAE 14.9
fold 4: train to 2025-10-02 (1357 days) → validate 2025-10-03–2025-12-31 | MAE 13.1
shuffled KFold MAE 12.8: optimistic, because it trains on the future
```

Hyperparameters

Parameters are learned from data (coefficients, tree splits). **Hyperparameters** are set before training (C, number of neighbours, tree depth, learning rate). We choose them by cross-validation.

GridSearchCV

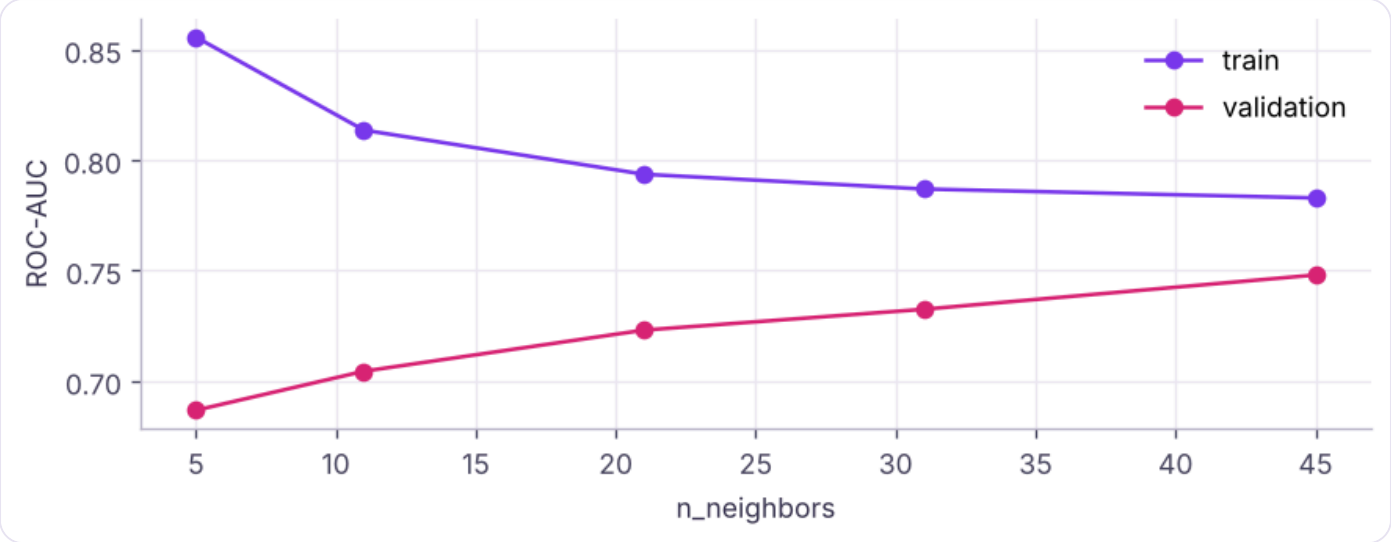
Python

```
from sklearn.neighbors import KNeighborsClassifier
knn = Pipeline([("scale", StandardScaler()), ("knn", KNeighborsClassifier())])
grid = {"knn__n_neighbors": [5, 11, 21, 31, 45], "knn__weights": ["uniform", "distance"],
"knn__p": [1, 2]}
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.25, stratify=y, random_state=7)
gs = GridSearchCV(knn, grid, cv=cv, scoring="roc_auc", n_jobs=-1,
return_train_score=True).fit(Xtr, ytr)
print("best:", gs.best_params_, f"| CV AUC {gs.best_score_:.3f} | test AUC
{roc_auc_score(yte, gs.predict_proba(Xte)[:, 1]):.3f}")
r = pd.DataFrame(gs.cv_results_)[["param_knn__n_neighbors", "param_knn__weights",
"param_knn__p", "mean_train_score", "mean_test_score", "rank_test_score"]]
print(r.sort_values("rank_test_score").head(6).round(3).to_string(index=False))
```

OUTPUT

```
best: {'knn__n_neighbors': 45, 'knn__p': 2, 'knn__weights': 'uniform'} | CV AUC 0.748
| test AUC 0.827
 param_knn__n_neighbors param_knn__weights  param_knn__p  mean_train_score
mean_test_score  rank_test_score
                     45            uniform             2             0.783
0.748                1
                     45            uniform             1             0.787
0.746                2
                     45           distance             2             1.000
0.745                3
                     45           distance             1             1.000
0.744                4
                     31            uniform             1             0.788
0.740                5
                     31           distance             1             1.000
0.734                6
```

Pipeline step names prefix parameters (`knn__n_neighbors`). `refit=True` (default) retrains the best configuration on all training data, so `gs` can predict directly.



Validation AUC by number of neighbours: small k overfits (train ≫ validation), large k smooths.

RandomizedSearchCV

Grids grow exponentially (5 values × 5 parameters = 3,125 fits per fold). Random search samples a fixed number of combinations and often finds equally good settings much faster.

Python

```
from scipy.stats import loguniform, randint
from sklearn.ensemble import RandomForestClassifier
rs = RandomizedSearchCV(RandomForestClassifier(random_state=0),
                        {"n_estimators": randint(100, 500), "max_depth": [None, 3, 5, 8,
12], "min_samples_leaf": randint(1, 30), "max_features": ["sqrt", 0.5, 1.0]},
                        n_iter=25, cv=cv, scoring="roc_auc", random_state=0,
n_jobs=-1).fit(Xtr, ytr)
print("best RF:", rs.best_params_, f"| CV AUC {rs.best_score_:.3f} | test AUC
{roc_auc_score(yte, rs.predict_proba(Xte)[:, 1]):.3f}")
lr_search = RandomizedSearchCV(make_pipeline(StandardScaler(),
LogisticRegression(max_iter=2000)), {"logisticregression__C": loguniform(1e-3, 1e2)},
n_iter=20, cv=cv, scoring="roc_auc", random_state=0).fit(Xtr, ytr)
print(f"logistic C = {lr_search.best_params_['logisticregression__C']:.3g} | CV AUC
{lr_search.best_score_:.3f} | test AUC {roc_auc_score(yte, lr_search.predict_proba(Xte)[:,
1]):.3f}")
```

OUTPUT

```
best RF: {'max_depth': 5, 'max_features': 'sqrt', 'min_samples_leaf': 24,
'n_estimators': 170} | CV AUC 0.776 | test AUC 0.847
logistic C = 0.441 | CV AUC 0.783 | test AUC 0.864
```

Simple, well-regularised logistic regression is hard to beat on this small, mostly linear credit dataset: a useful reminder to always include simple models in the comparison.

Bayesian optimisation with Optuna

Python

```
import optuna
def objective(trial):
    params = {"n_estimators": trial.suggest_int("n_estimators", 100, 800),
              "max_depth": trial.suggest_int("max_depth", 2, 12),
              "min_samples_leaf": trial.suggest_int("min_samples_leaf", 1, 40),
              "max_features": trial.suggest_float("max_features", 0.2, 1.0)}
    return cross_val_score(RandomForestClassifier(**params, random_state=0), Xtr, ytr,
cv=cv, scoring="roc_auc").mean()
study = optuna.create_study(direction="maximize")
study.optimize(objective, n_trials=60)
print(study.best_params, study.best_value)
```

Optuna (and scikit-optimize, Hyperopt) learns from earlier trials which regions look promising, and prunes bad trials early: the usual choice for gradient boosting and deep learning.

Nested cross-validation

The best CV score from a search is slightly optimistic: it was chosen as the maximum of many tries. **Nested CV** wraps the search in an outer loop for an unbiased estimate.

Python

```
inner = StratifiedKFold(5, shuffle=True, random_state=1); outer = StratifiedKFold(5,
shuffle=True, random_state=2)
search = GridSearchCV(knn, {"knn__n_neighbors": [5, 11, 21, 31, 45]}, cv=inner,
scoring="roc_auc")
nested = cross_val_score(search, X, y, cv=outer, scoring="roc_auc")
print(f"nested CV AUC {nested.mean():.3f} ± {nested.std():.3f} (vs best inner score
{search.fit(X, y).best_score_:.3f})")
```

OUTPUT

```
nested CV AUC 0.783 ± 0.044 (vs best inner score 0.786)
```

Leakage checklist

Leak	Example	Prevention
Preprocessing on all data	Scaling or imputing before splitting	Put every step in a Pipeline
Target leakage	A feature recorded after the outcome	Feature cut-off dates
Duplicate entities	Same customer in train and validation	GroupKFold
Future data	Random splits for time series	TimeSeriesSplit
Tuning on test data	Picking the model with the best test score	Tune with CV; test once
Feature selection outside CV	SelectKBest on all rows	Selection inside the pipeline

Summary

KEY TAKEAWAYS

- One split is noisy; k-fold CV averages several splits; report mean ± spread.
- Use stratified folds for classification, groups for repeated entities, and time-series splits for temporal data.
- Hyperparameters are tuned by CV: GridSearchCV (small spaces), RandomizedSearchCV (large spaces), Optuna (efficient Bayesian search).
- Nested CV gives an unbiased estimate of a tuned model; the test set is still used only once.
- Keep every data-dependent step inside the pipeline to avoid leakage.

Quick quiz

6 questions · tap an answer to check it

1. Why use cross-validation instead of one train/test split?

- **A** A single split is noisy; averaging over folds gives a more reliable estimate
- **B** It makes models more accurate
- **C** It avoids needing a test set
- **D** It is faster

Answer: A. Especially important for small datasets.

2. Which splitter for daily sales forecasting?

- **A** StratifiedKFold
- **B** TimeSeriesSplit
- **C** Shuffled KFold
- **D** LeaveOneOut

Answer: B. Validation must come after training in time.

3. How do you name a hyperparameter of a pipeline step called "knn" in GridSearchCV?

- **A** n_neighbors
- **B** knn__n_neighbors
- **C** knn.n_neighbors
- **D** knn-n_neighbors

Answer: B. Step name, two underscores, parameter.

4. When is RandomizedSearchCV preferable to GridSearchCV?

A When the search space is large; it samples a fixed number of combinations

B When there is one hyperparameter

C Never

D Only for regression

Answer: A. Often as good, much cheaper.

5. What does nested cross-validation estimate?

A The performance of the whole tuning procedure, without optimistic bias

B The best hyperparameters only

C Training time

D Feature importance

Answer: A. Inner loop tunes, outer loop evaluates.

6. Scaling the full dataset before cross-validation causes...

A Leakage: validation statistics influence training

B Better accuracy legitimately

C No effect

D Slower training

Answer: A. Put the scaler in the pipeline.

EXERCISE

Practice

(1) Tune `C` and `penalty` (`l1`, `l2`) for logistic regression on the bank data with GridSearchCV (solver `liblinear`). (2) Use `GroupKFold` on the superstore data so that all lines of one customer stay together, predicting whether a line is loss-making. (3) Plot the CV AUC for `max_depth` 1–15 of a decision tree.

Show solution (worked answer)

Python

```
g = GridSearchCV(make_pipeline(StandardScaler(),
LogisticRegression(solver="liblinear")), {"logisticregression__C": np.logspace(-3, 2,
12), "logisticregression__penalty": ["l1", "l2"]}, cv=cv, scoring="roc_auc").fit(Xtr,
ytr)
print(g.best_params_, g.best_score_)
from sklearn.model_selection import GroupKFold
from sklearn.tree import DecisionTreeClassifier
s = pd.read_csv("superstore_sales.csv"); Xs = pd.get_dummies(s[["sales", "quantity",
"discount", "category", "region"]]); ys = (s.profit < 0).astype(int)
print(cross_val_score(DecisionTreeClassifier(max_depth=5), Xs, ys,
groups=s.customer_id, cv=GroupKFold(5), scoring="roc_auc").mean())
print([round(cross_val_score(DecisionTreeClassifier(max_depth=k, random_state=0), X,
y, cv=cv, scoring="roc_auc").mean(), 3) for k in range(1, 16)])
```

INTERVIEW CORNER

Questions you may be asked

Q: Explain k-fold cross-validation. Split the training data into k folds; train on k − 1 and validate on the remaining one, k times; average the scores. It uses all data for both training and validation and reduces variance of the estimate.

Q: Grid search vs random search vs Bayesian optimisation? Grid tries every combination (exhaustive, expensive); random samples combinations (efficient in high dimensions); Bayesian optimisation models the score surface to choose promising settings next.

Q: How would you validate a churn model that will score customers monthly? Time-based validation: train on earlier months and validate on later months, mimicking deployment; also group by customer to avoid leakage.

Practise more online

Free, trusted places to test yourself on this topic. Links open the original site in a new tab.